Big-O, bất biến và kiểm tra thuật toán

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 3: Phân tích thuật toán và đệ quy.
- Các bài trong chương: Big-O, bất biến và kiểm tra thuật toán; Đệ quy và ngăn xếp lời gọi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Phân tích chi phí theo kích thước đầu vào và nêu điều kiện để thuật toán đúng.

Kiến thức nền

Kích thước đầu vào n cần được xác định trước khi phân tích. Một vòng lặp thăm n phần tử thường O(n); hai vòng lặp lồng nhau, mỗi vòng n bước, thường $O(n^2)$. Hai vòng nối tiếp có chi phí O(n)+O(n), vẫn là O(n). Vòng lặp chia đôi miền mỗi bước thường gợi ý O(log n).

Phân biệt trường hợp tốt nhất, xấu nhất và trung bình. Big-O là chặn trên tiệm cận; hằng số, kiểu dữ liệu và môi trường vẫn ảnh hưởng thời gian chạy thực tế. Bộ nhớ phụ phân tích riêng với bộ nhớ chứa đầu vào.

Bất biến là mệnh đề đúng tại các mốc trong vòng lặp, giúp lập luận đúng đắn. Khi tìm số lớn nhất, sau k bước, biến best phải chứa giá trị lớn nhất trong k phần tử đã xét. Test rỗng, một phần tử, trùng lặp và biên giúp tìm lỗi; một vài test qua chưa chứng minh đúng với mọi đầu vào.

1. Đếm trước khi rút gọn

```
for i in range(n): pass          -> n bước
Hai vòng trên nối tiếp           -> 2n bước -> O(n)
Hai vòng n bước lồng nhau        -> n*n bước -> O(n²)
while n > 1: n //= 2             -> khoảng log2(n) bước
```

Nếu vòng trong không chạy n lần mà phụ thuộc i, phải viết tổng thực tế. Với j từ 0 tới i-1, số bước là n(n-1)/2, vẫn $O(n^2)$. Không suy ra cứ thấy hai vòng là chắc $O(n^2)$.

2. Lập luận bằng bất biến

```
def maximum(values):
    if not values:
        raise ValueError('empty input')
    best = values[0]
    for item in values[1:]:
        if item > best:
            best = item
    return best

assert maximum([-9, -2, -5]) == -2
assert maximum([7]) == 7
```

Khởi tạo: best đúng cho tiền tố dài một. Duy trì: so sánh phần tử mới giữ max của tiền tố đã xét. Kết thúc: tiền tố là toàn mảng. Bản dùng slicing ở ví dụ tạo danh sách phụ; nếu phân tích bộ nhớ phải tính chi phí này hoặc chuyển sang lặp chỉ số.

3. Thiết kế test theo đặc tính

Kết quả max phải thuộc dữ liệu và không nhỏ hơn mọi phần tử. Các đặc tính giúp chọn test rỗng, âm, lặp, tăng và giảm. Test chạy qua không thay thế chứng minh; chứng minh cũng không bảo đảm cài đặt không có lỗi gõ.

4. Bài tập và đáp án

Tính tổng từ i=0 đến n-1 của i: n(n-1)/2. Giải thích khác nhau giữa O(n), Θ(n) và một phép đo giây: chặn trên, chặn khít tiệm cận và kết quả thực nghiệm. Nếu chỉ biết thuật toán $O(n^2)$, không tự kết luận đó là chặn khít tốt nhất.

> ### Bài tập vận dụng bổ sung
>
> Phân tích hai vòng nối tiếp và hai vòng lồng nhau. Viết hàm tìm max có quy ước khi đầu vào rỗng; thử [-5,-2,-9], [7], [2,2].

Hướng giải bài tập bổ sung

Chi phí lần lượt O(n) và $O(n^2)$. Với mảng rỗng phải trả dấu hiệu không có kết quả hoặc báo lỗi đã quy định, không tự trả 0.

Câu hỏi tự kiểm tra

1. Hai vòng n bước chạy nối tiếp thường có Big-O nào?
2. Hai vòng n bước lồng nhau thường có chi phí nào?
3. Bộ nhớ phụ nên được đánh giá thế nào?
4. Khởi tạo max bằng 0 dễ sai với dữ liệu nào?

5. Bất biến vòng lặp dùng để làm gì?

Gợi ý đáp án và điểm cần nhớ

- O(n). Hệ số 2 được bỏ trong biểu diễn tiệm cận.
- $O(n^2)$. Tổng số lần thực hiện phần thân thường là n nhân n.
- Riêng với chi phí thời gian. Thuật toán có thể nhanh hơn nhưng dùng nhiều bộ nhớ hơn.
- Mảng chỉ có số âm. 0 không thuộc mảng và lớn hơn mọi phần tử âm.
- Hỗ trợ lập luận trạng thái và tính đúng đắn. Bất biến mô tả điều được giữ đúng trong quá trình lặp.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Big-O, bất biến và kiểm tra thuật toán — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: hai vòng lặp không phải lúc nào cũng $O(n^2)$. Hai vòng nối tiếp mỗi vòng n bước cho O(n); vòng lồng mỗi vòng n bước thường cho $O(n^2)$ theo mô hình công việc mỗi bước hằng số.

Tự luyện: đếm số phép so sánh của tìm tuyến tính trên đầu, giữa, cuối và không tìm thấy. **Hướng giải:** nêu n là số phần tử, không phải "độ dài code". Bất biến giúp chứng minh: sau k phần tử đã duyệt, tổng tạm bằng tổng đúng k phần tử đó. Kiểm thử có thể tìm phản ví dụ nhưng không thay thế mọi chứng minh tổng quát.

Nguồn học tiếp

https://docs.python.org/3/library/collections.html